

OWASP A03 – Injection

1 What Is Injection?

Injection happens when:

An application **blindly trusts user input** and sends it to an interpreter (SQL, OS, LDAP, etc.).

- The system **executes the input instead of treating it as data.**

2 Why Injection Is So Dangerous

Because it can lead to:

- Database data theft
- Login bypass
- Data modification
- Full server takeover (in command injection)

Note : Injection = **Loss of trust boundary**

3 Conceptual Attack Flow

Phase 1: Input Point Discovery

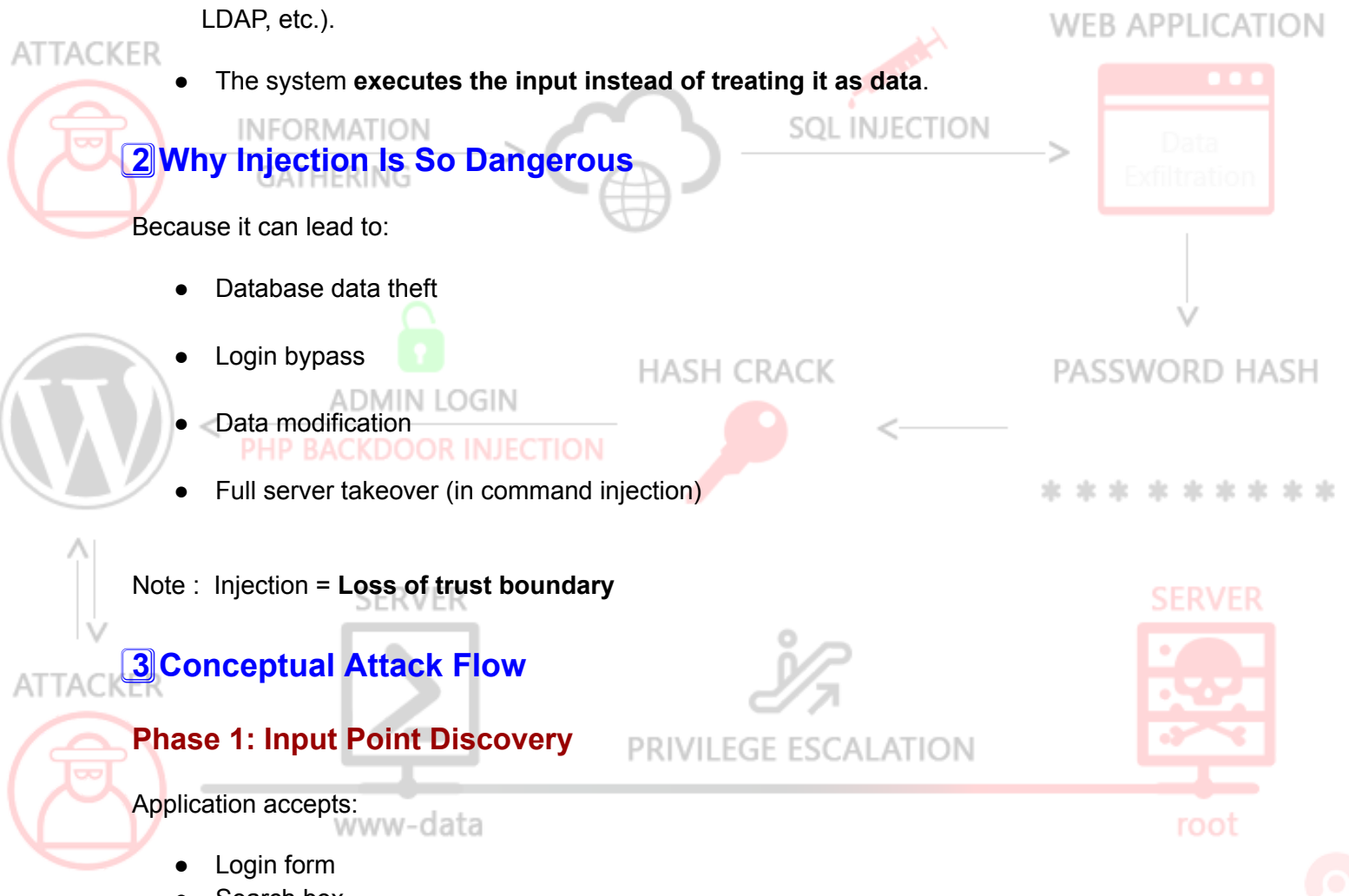
Application accepts:

- Login form
- Search box
- URL parameter
- API request

Example:

`/product?id=10`

ATTACK OVERVIEW



Phase 2: Trust Boundary Violation

Backend code (bad example):

```
SELECT * FROM products WHERE id = '$user_input';
```

✗ User input is **directly inserted** into SQL

Phase 3: Interpreter Confusion

Database cannot distinguish:

- Code
- Data

So attacker input **changes query logic**

✗ This is NOT “breaking the DB”
It is **tricking the app into issuing a dangerous query**

4 Example Scenario

A website has a login page.

User enters:

- Username
- Password

1 Normal (Expected) SQL Query

Backend SQL (what developer wants)

- `SELECT * FROM users`
- `WHERE username = 'raj'`
- `AND password = 'mypassword';`

Meaning:

“Check if there is a user named **raj with this password.**”

Output (Normal Case): 1 row found

- Login successful

2 Where the Problem Starts (Bad Coding)

Vulnerable Code Logic

- `SELECT * FROM users`
- `WHERE username = '<USER_INPUT>'`
- `AND password = '<USER_INPUT>';`

⚠ Problem:

User input is directly placed into SQL query
No validation, no parameterization.

3 Injection Concept (What Goes Wrong)

Instead of input being treated as data,
the database treats it as SQL logic.

Database gets confused:

- “Is this data?”
- “Or is this a command?”

That confusion = Injection

4 Conceptual Injection Example (SAFE)

User input (conceptual)

- Username: [input that changes query logic]
- Password: anything

Resulting SQL (logic changed)

- `SELECT * FROM users`
- `WHERE username = 'something'`
- `AND condition becomes always true;`

- ✚ You don't need to know the exact payload
- ✚ Just understand: logic of query changes

5 What the Application Sees (Output)

Instead of:

- 0 rows found
- Login failed

The app sees:

- Multiple rows found
- Login successful

! This means:

- Authentication is bypassed
- Database trusted wrong logic



6 Example – Product Page

URL

- /product?id=10

Backend SQL

- `SELECT * FROM products WHERE id = 10;`

Vulnerable Logic

- `SELECT * FROM products WHERE id = <USER_INPUT>;`

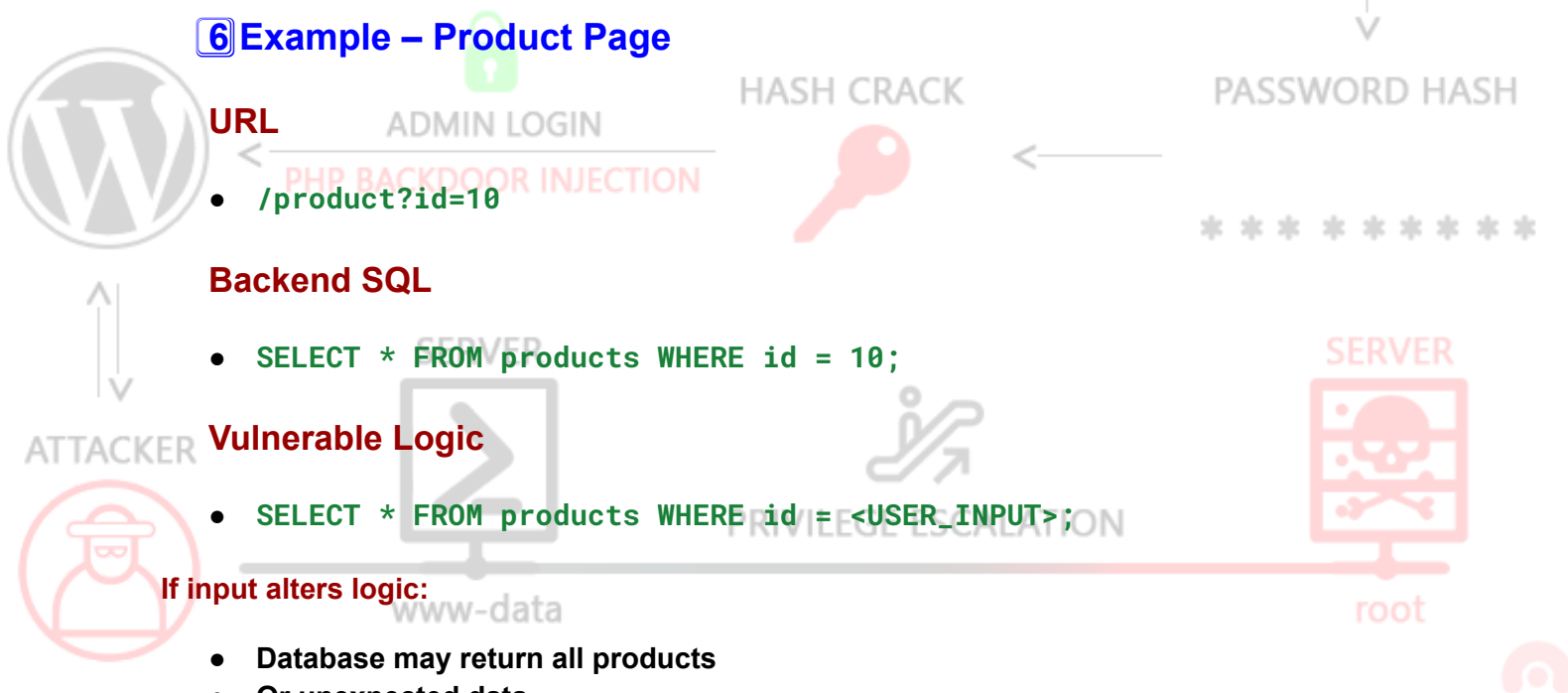
If input alters logic:

- Database may return all products
- Or unexpected data

Output Seen by Tester

- Product list instead of single product

✚ That abnormal output is evidence of injection



7 Error-Based Example (Very Common)

Normal Output

- Product details shown

After bad input

- SQL syntax error near '...'

📌 This error means:

- SQL engine is processing user input
- Input reached the database layer
- Injection risk exists

ATTACKER



INFORMATION
GATHERING



SQL INJECTION

WEB APPLICATION



PASSWORD HASH

8 Impact

Because of SQL Injection:

- User data leaked
- Passwords stolen
- Orders modified
- Full database dumped

ADMIN LOGIN

HASH CRACK



* * * * *

📌 Many real breaches happened this way, not because of “advanced hacking”.

9 How Secure Code Fixes This

✗ Vulnerable Query

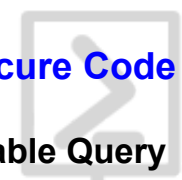
- `SELECT * FROM users WHERE username = 'input';`

✓ Secure Query (Parameterized)

- `SELECT * FROM users WHERE username = ?;`

Note : SQL Injection = User input controls SQL logic

ATTACKER



PRIVILEGE ESCALATION

SERVER



root